

# 编译技术实验

杨海燕

2026年10月8日




**2026全国大学生计算机系统能力大赛**
**编译系统设计赛（华为毕昇杯）**
**全国总决赛获奖名单（挑战赛道）**

| 赛道 | 队伍名称 | 学校 | 队员 | 奖项 |
|---|---|---|---|---|
| 挑战赛-面向LoongArch架构的二进制优化与布局工具 | 译路领航 | 北京航空航天大学 | 罗天翼 郭蕙源 李岳暄 | 一等奖 |
| 挑战赛-面向LoongArch架构的二进制优化与布局工具 | For_Super_Earth | 北京航空航天大学 | 罗文烨 马浩然 | 三等奖 |
| 挑战赛-面向LoongArch架构的二进制优化与布局工具 | 郑青春 | 郑州大学 | 马驰尧 杨洋 胡昊宇 孟凡兹 | 三等奖 |
| 挑战赛-面向LoongArch架构的二进制优化与布局工具 | 冷月葬花魂 | 江南大学 | 杨梓 郭嘉杨 周政汐 郑诗哲 | 三等奖 |

**2026全国大学生计算机系统能力大赛**
**编译系统设计赛（华为毕昇杯）**
**全国总决赛获奖名单（ARM赛道）**

| 赛道 | 队伍名称 | 学校 | 队员 | 奖项 |
|---|---|---|---|---|
| ARM | 都能队 | 北京师范大学 | 周飞翼 张亦法 钟凯熙 王若涵 | 一等奖 |
| ARM | NO_COMPILE_NO_LIFE | 华中科技大学 | 李桦 丁琦扬 丁树浩 赵凯峰 | 一等奖 |
| ARM | Macrohard | 复旦大学 | 佟铭洋 | 二等奖 |
| ARM | 未定义行为队 | 北京航空航天大学 | 王琦 肖一涵 刘笑晨 任一杰 | 二等奖 |
| ARM | CompilersLegion | 南京大学 | 缪宇轩 石城玮 沈蔡子睦 吴宸暄 | 三等奖 |
| ARM | 独断万古荒天帝 | 武汉大学 | 张开宇 杨俊 赵宇恒 | 三等奖 |
| ARM | 变异のCompiler | 南开大学 | 王为 牛帅 林盛森 陈兴浩 | 三等奖 |
| ARM | 自在极译编 | 武汉大学 | 陈秋雨 | 优胜奖 |
| ARM | Ph4ot0m_Opt | 中国科学技术大学 | 李培锋 王玺玮 | 三等奖 |
| ARM | 北京阿侬编译技术有限公司 | 北京理工大学 | 左天佑 蒋浩天 张振宇 | 三等奖 |
| ARM | Vortex | 复旦大学 | 王思宇 李英稷 洪昭晖 | 三等奖 |
| ARM | AAA编译器批发 | 南开大学 | 王众 许洋 周重天 | 三等奖 |
| ARM | 编的队 | 北京航空航天大学 | 李奕宽 陈润 传真奇 | 三等奖 |

**2026全国大学生计算机系统能力大赛**
**编译系统设计赛（华为毕昇杯）**
**全国总决赛获奖名单（RISC-V赛道）**

| 赛道 | 队伍名称 | 学校 | 队员 | 奖项 |
|---|---|---|---|---|
| RISC-V | 未闻队名 | 南京大学 | 崔可喻 郭佳荣 熊雅琪 程心妍 | 一等奖 |
| RISC-V | 规约珞樱 | 武汉大学 | 王仕杰 吴润康 | 一等奖 |
| RISC-V | Mars | 北京航空航天大学 | 陈耕 田锦煜 朱永川 | 二等奖 |
| RISC-V | wsgd | 杭州电子科技大学 | 黄可染 王佳栋 | 二等奖 |
| RISC-V | TACocat | 电子科技大学 | 魏子越 陈筑江 姜俊吉 李智捷 | 二等奖 |
| RISC-V | GirlsBandCompiler | 西安交通大学 | 余凯越 陶泽睿 许佳骏 | 二等奖 |
| RISC-V | 汇芯一笑 | 电子科技大学（沙河校区） | 刘烨嘉一 宋科宏 | 二等奖 |
| RISC-V | 那就全队 | 国防科技大学 | 谭思平 吴浩 龚妍颖 吴君和 | 二等奖 |
| RISC-V | 队伍名称 | 北京航空航天大学 | 王腾扬 赵栩栩 卢一非 | 三等奖 |

# 课程目标

- 掌握编译系统的功能、原理和构造方法（不局限于特定的程序设计语言和目标机）；理解高级程序设计语言的运行机理。
- 具备完整小型编译系统的设计、实现和测试的能力；理解一种程序设计语言程序到另一种程序等价转化的理论和方法。
- 掌握基本优化算法，具备针对特定目标体系架构生成目标代码的能力；理解程序运行的安全和性能问题。
- 掌握形式语言以及自动机理论在编译器前端自动生成中的应用；理解不同方法和工具的作用和局限性。

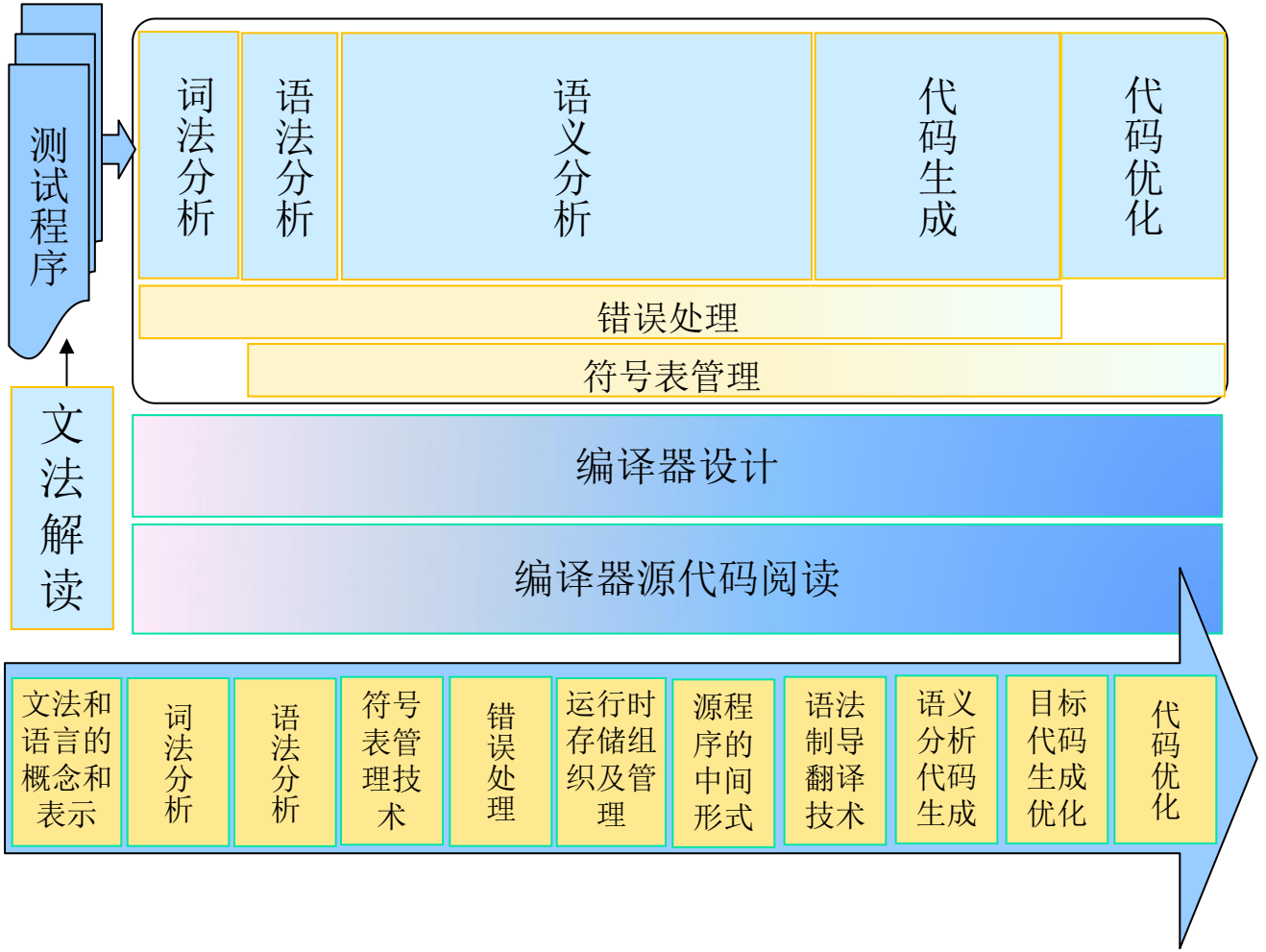
| | | |
|---|---|---|
| 能将专业知识和数学分析方法用于推演、分析计算机领域复杂工程问题 | 能够针对计算机领域复杂工程问题的特定需求，完成系统、单元的设计 | 能够根据计算机领域的对象特征，选择研究路线，设计研究方案，构建系统，采集相关数据 |

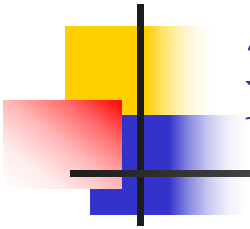
# 三次学习编译全过程

- 第一次：概述
  - 大致了解编译的过程和编译程序的构造
- 第二次：第3-10，14，15章
  - 详细学习各部分的原理和方法
- 第三次：实验
  - 基于理论学习，逐步实现一个完整编译器



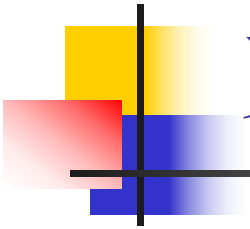
# 理论课与实验作业概览


测试程序
词法分析
语法分析
语义分析
代码生成
代码优化
错误处理
符号表管理
文法解读
编译器设计
编译器源代码阅读
文法和语言的概念和表示
词法分析
语法分析
符号表管理技术
错误处理
运行时存储组织及管理
源程序的中间形式
语法制导翻译技术
语义分析代码生成
目标代码生成优化
代码优化



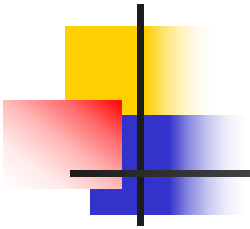
# 实验题目

- 根据给定的文法和要求实现编译器
  - 文法（语法定义、语义约定）
  - 中间代码
  - 目标码
  - 优化



# 文法

- SysY语言简化版
- 如无特殊说明，语义参照C语言
  - 算术运算不进行整型提升
- 具有常量、变量、整数、字符、字符串、一维数组、函数（带参数）、赋值语句、if语句、while语句、switch语句、break语句、continue语句、语句块、输入输出语句等



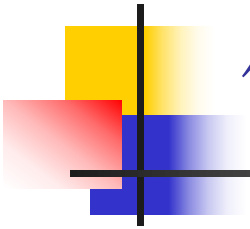
# 中间代码

- 按一定要求自行设计的四元式
- 也可以用LLVM IR作为中间代码
- 生成PCODE时可以没有中间代码



# 目标代码（三选一）

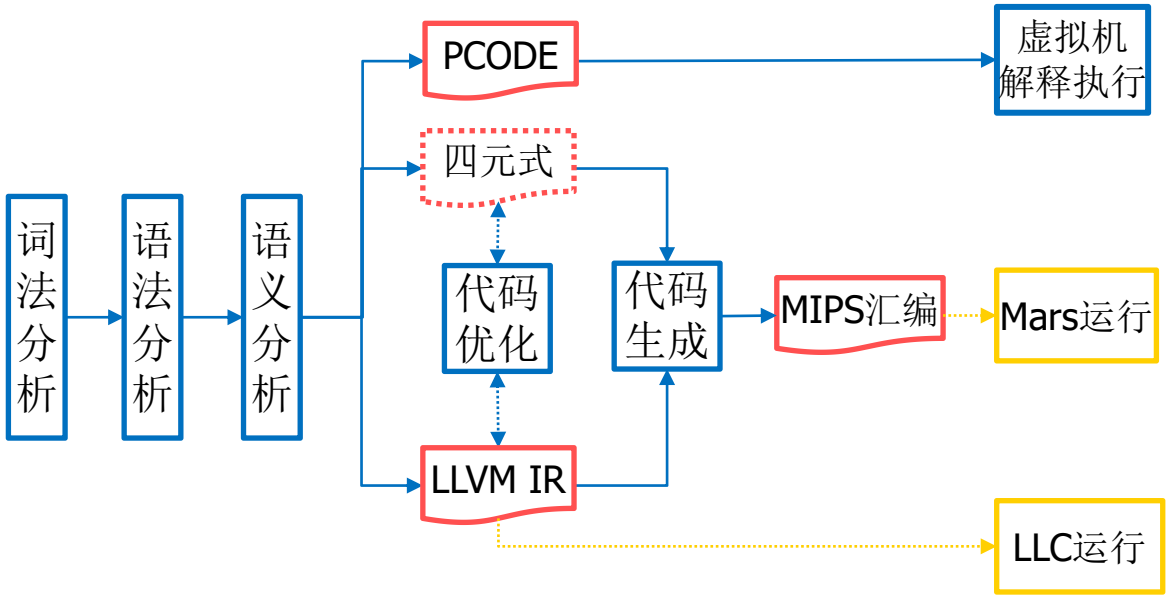
- PCODE
  - 可参照PASCAL-S编译器的设计
  - 需编写解释执行程序对PCODE代码解释执行
- LLVM IR
  - 完成load/store形式的LLVM IR
  - LLVM IR的运行由llc工具完成
- MIPS汇编
  - 生成32位MIPS汇编码
  - 代码生成时合理利用临时寄存器（临时寄存器池），并能生成较高质量的目标代码



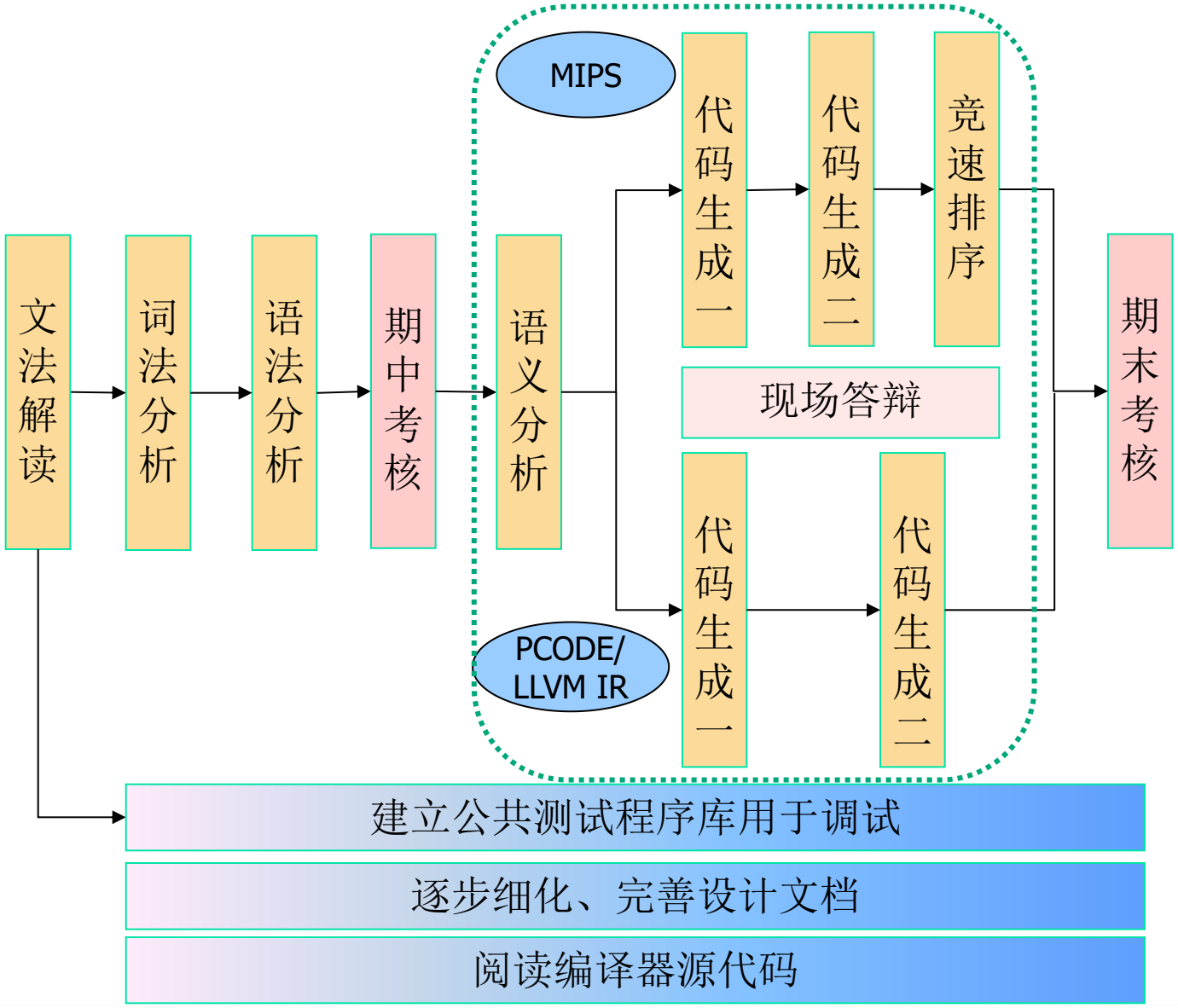
# 代码优化

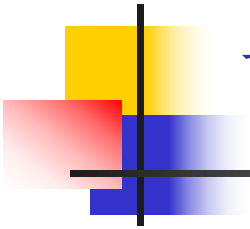
- 基本块内部的公共子表达式删除（DAG图）；
- 全局寄存器分配（引用计数或着色算法）；
- 数据流分析（通过活跃变量分析，或利用定义-使用链建网等方法建立冲突图）；
- 其它优化自选

# 任务架构


词法分析
语法分析
语义分析
PCODE
四元式
代码优化
LLVM IR
代码生成
MIPS汇编
虚拟机解释执行
Mars运行
LLC运行

# 任务及考核步骤


文法解读
词法分析
语法分析
期中考核
语义分析
MIPS
代码生成一
代码生成二
竞速排序
现场答辩
PCODE/
LLVM IR
代码生成一
代码生成二
期末考核
建立公共测试程序库用于调试
逐步细化、完善设计文档
阅读编译器源代码



# 评分标准

- 生成PCODE解释执行：最高85分
- 生成LLVM IR：最高85分
- 生成MIPS汇编：最高分100
  - 参加竞速排序
  - 完成优化文章

# 分数组成

实验成绩的100分由以下部分组成：

| 文法解读 | 词法分析 | 语法分析 | 语义分析 | 代码生成一 | 代码生成二 | 代码优化 | 文档 | 期中考核 | 期末考核 |
|---|---|---|---|---|---|---|---|---|---|
| 5 | 5 | 5 | 5 | 5 | 15 | 15 | 5 | 15 | 25 |

其中代码优化的15分由以下部分组成：

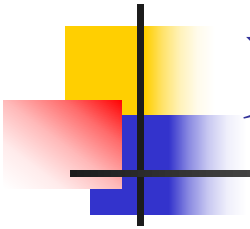
| 竞速排序作业 | 期末考核优化题目 | 优化文章 |
|---|---|---|
| 10 | 3 | 2 |

文档的5分由以下部分组成：

| 设计文档 | 总结感想 |
|---|---|
| 3.5 | 1.5 |

# 关于分数的注意事项

- 只有当代码生成二作业、竞速排序作业和期末考核优化题目都提交了生成MIPS的版本，才能得到代码优化的分数。上述3项缺少任一项都不能得到代码优化的分数，这种情况下作业的最高分是85分
- 竞速排序作业中的题目"优化前后中间代码目标代码提交"，满分是0分，发现问题会扣分
- 现场答辩的满分是0分，发现问题会扣分



# 文法解读

- 仔细阅读文法，对文法中每条规则所定义的语法成分进行分析，了解其作用、限定条件、组合情况和可能产生的出句子
- 编写6个测试程序，要求测试程序能覆盖所有语法规则与常见的组合
- 每个测试程序有多行输出结果
  - 第一条写语句请输出自己的学号
  - 其他写语句需尽量反映出程序定义的数据及其运算结果
- 文档中列出的需覆盖项只是最基本的

# 文法解读

- 请提供每个测试程序的输入数据(有<读语句>则提供，否则只需提供空的输入文件)、输出数据（若输入输出数据没有正确提供，评测时会报错），放到文件中，按下述要求为文件命名：

  测试程序及对应的输入输出数据文件分别为

testfile1.txt　input1.txt　output1.txt
testfile2.txt　input2.txt　output2.txt
...
testfile6.txt　input6.txt　output6.txt

# 文法解读

- 2个用例需包含A级规则
- 2个用例需包含B级规则，不能含有A级规则
- 2个用例需包含C级规则，不能含有A、B级的规则



文法解读评测结果

| 本次评测得分比例 | 覆盖项总数 | 覆盖项覆盖数 | 最高重复率 |
|---|---|---|---|
| 100% | 95 | 95 | 46.01% testfile2.txt 与 testfile3.txt |

测试用例

| ID | 文件 | 评测结果 | 程序等级 | 输入/输出详情 | 编译器输出 / 错误信息 | 覆盖项覆盖数 |
|---|---|---|---|---|---|---|
| 1 | testfile1.txt | Accepted | C | 查看输入/输出详情 | 无 | 53 |
| 2 | testfile2.txt | Accepted | C | 查看输入/输出详情 | 无 | 60 |
| 3 | testfile3.txt | Accepted | B | 查看输入/输出详情 | /tmp/for_super_judge1239110806735852477/testfile3.c:17:26: warning: ISO C forbids empty initializer braces [-Wpedantic]<br>17 \| const int empty_arr[3] = {};<br>\| ^<br>/tmp/for_super_judge1239110806735852477/testfile3.c: In function 'main':<br>/tmp/for_super_judge1239110806735852477/testfile3.c:43:22: warning: ISO C forbids empty initializer braces [-Wpedantic]<br>43 \| int nil_arr[2] = {};<br>\| ^ | 58 |



文法解读评测结果

| 本次评测得分比例 | 覆盖项总数 | 覆盖项覆盖数 | 最高重复率 |
|---|---|---|---|
| 94% | 95 | 89 | 100.00% testfile4.txt 与 testfile3.txt |

测试用例

| ID | 文件 | 评测结果 | 程序等级 | 输入/输出详情 | 编译器输出 / 错误信息 | 覆盖项覆盖数 |
|---|---|---|---|---|---|---|
| 1 | testfile1.txt | Accepted | A | 查看输入/输出详情 | GCC Compiler Output:<br>Reference Compiler Output: | 69 |
| 2 | testfile2.txt | Accepted | A | 查看输入/输出详情 | GCC Compiler Output:<br>Reference Compiler Output: | 86 |
| 3 | testfile3.txt | Accepted | B | 查看输入/输出详情 | GCC Compiler Output:<br>Reference Compiler Output: | 27 |
| 4 | testfile4.txt | Accepted | B | 查看输入/输出详情 | GCC Compiler Output: | 27 |

# 词法分析

- 有统一的类别码定义

| 单词名称 | 类别码 | 单词名称 | 类别码 | 单词名称 | 类别码 | 单词名称 | 类别码 |
|---|---|---|---|---|---|---|---|
| **Ident** | IDENFR | while | WHILETK | >= | GEQ | ; | SEMICN |
| **IntConst** | INTCON | case | CASETK | \|\| | OR | < | LSS |
| **CharConst** | CHARCON | char | CHARTK | ! | NOT | = | ASSIGN |
| **StringConst** | STRCON | else | ELSETK | % | MOD | > | GRE |
| continue | CONTINUETK | main | MAINTK | ( | LPARENT | [ | LBRACK |
| default | DEFAULTTK | void | VOIDTK | ) | RPARENT | ] | RBRACK |
| printf | PRINTFTK | int | INTTK | * | MULT | { | LBRACE |
| return | RETURNTK | if | IFTK | + | PLUS | } | RBRACE |
| static | STATICTK | != | NEQ | , | COMMA | - | MINU |
| break | BREAKTK | <= | LEQ | / | DIV | switch | SWITCHTK |
| const | CONSTTK | == | EQL | : | COLON | && | AND |

# 词法分析

- 输入的被编译源文件统一命名为 testfile.txt；输出的结果文件统一命名为 lexer.txt
- 按顺序和格式输出类别码和单词字符串形式

单词类别码 单词的字符/字符串形式(中间仅用一个空格间隔)

testfile.txt

```
int main() {
    int a = 65;
    char b = (char)a;
    put_char(b);
    printf("\n");

    char c = 'A';
    int d = (int)c;
    put_int(d);
    printf("\n");

    int e = 300;
    char f = (char)e;
    put_int((int)f);
    printf("\n");
    return 0;
}
```

lexer.txt

INTTK int
MAINTK main
LPARENT (
RPARENT )
…

# 词法分析

- 有**非法单词**时输出至error.txt
- 输出非法单词所在的行号以及错误类别码

testfile.txt

```
int main(){
    if(1 & 2){ // 非法单词 &
        printf("2026 Compiler\n");
    }
    return 0;
}
```

error.txt

2 a

# 语法分析

- 输入的被编译源文件统一命名为 testfile.txt
- 在词法分析程序输出的基础上，输出特定语法成分的名字（非终结符）至parser.txt中

testfile.txt

```
int main(){
    int c;
    c= get_int();
    printf("%d",c);
    return c;
}
```

parser.txt

INTTK int
MAINTK main
LPARENT (
RPARENT )
LBRACE {
INTTK int
IDENFR c
<VarDef>
SEMICN ;
<VarDecl>
…

# 语法分析

- 有**非法单词**和**语法错误**时输出出错行号和错误类别码至 error.txt

testfile.txt

```
int main(){
    int c;
    c = get_int()
    if(1 & 2){
        printf("output is %d\n", c);
    }
    return 0;
}
```

error.txt

3 i
4 a

# 语义分析

- 输入的被编译源文件统一命名为 testfile.txt；
- 对正确的源程序，输出编译完成后的**符号表主要信息至**symbol.txt

testfile.txt

```
const int year = 2026, month =
int day;

int getDay(){
    static int day;
    day = get_int();
    return day;
}

int fun(){
    return 12345;
}
```

```
int main(){
    day = getDay();
    printf("Tody is %d-%d-%d\n", year, month, day);

    {
        int length = fun();
        if(length > 4){
            printf("%d\n", length);
        }
    }

    return 0;
}
```

symbol.txt

```
1 year ConstInt
1 month
ConstInt
1 day Int
1 getDay
IntFunc
1 fun IntFunc
2 day StaticInt
5 length Int
```

# 语义分析

- 在语法错误的基础上增加**语义错误；**
- 错误的源程序，输出出错行号和错误类别码至error.txt

testfile.txt

```
const int const1 = 1, const2 = -100;
int change1;
int gets1(int var1,int var2){
  const1 = 999;
  change1 = var1 + var2          return (change1);
}
int main(){
  change1 = 10;
  printf("Hello World$");
  return 0;
}
```

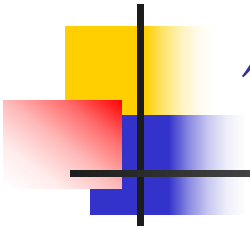
error.txt

4 h
5 i

# 错误类型及说明

| 错误类型 | 错误类别码 | 解释 | 对应文法及出错符号(...省略该条规则后续部分 |
|---|---|---|---|
| 非法符号 | a | 出现了 '&' 和 '\|' 这两个符号，应该将其当做 '&&' 与 '\|\|' 进行处理，报错行号为 '&' 或 '\|' 所在的行号。 | LAndExp → LAndExp '&&'EqExp<br>LOrExp → LOrExp '\|\|'LAndExp |
| 名字重定义 | b | 函数名或者变量名在**当前作用域**下重复定义。注意，变量一定是同一级作用域下才会判定出错，不同级作用域下，内层会覆盖外层定义。报错行号为**<Ident>**所在行数。 | ConstDef → Ident ...<br>VarDef → ... Ident ... Ident ...<br>FuncDef → FuncType Ident ...<br>FuncFParam → BType Ident ... |
| 未定义的名字 | c | 使用了未定义的标识符报错行号为**<Ident>**所在行数。 | LVal → Ident ...<br>UnaryExp → Ident ... |
| 函数参数个数不匹配 | d | 函数调用语句中，参数个数与函数定义中的参数个数不匹配。报错行号为函数调用语句的**函数名**所在行数。 | UnaryExp → Ident '(' [FuncRParams ] ')' |
| 类型不匹配 | e | 当表达式中出现多类型混合运算时，或者有返回值的函数返回的类型与定义的返回类型不一致，或者赋值语句中左值和右值类型不一致，或者函数参数传递时实参类型与形参类型不一致等情况时，报此错误。报错行号为发生类型不匹配的位置所在行号。 | Stmt → LVal '=' Exp ';'<br>Stmt → 'return' [Exp]';'<br>UnaryExp → Ident '('[FuncRParams] ')'<br>CaseStmt → 'case' Number':' ...<br>Stmt →'printf''('StringConst{','Exp}')'';' |
| 无返回值的函数存在不匹配的return语句 | f | 报错行号为**'return'**所在行号。 | Stmt → 'return' { '[' Exp ']' } ';' |
| 有返回值的函数缺少return语句 | g | 只需要考虑函数末尾是否存在return语句，**无需考虑数据流**。报错行号为函数**结尾的'}'**所在行号。 | FuncDef → FuncType Ident '('[FuncFParams] ')' Block<br>MainFuncDef → 'int' 'main' '(')' Block |
| 不能改变常量的值 | h | <LVal>为常量时，不能对其修改。报错行号为**<LVal>**所在行号。 | Stmt → LVal '=' Exp ';'<br>ForStmt → LVal '=' Exp ... |
| 缺少分号 | i | 报错行号为分号**前一个非终结符**所在行号。 | Stmt, ConstDecl 及 VarDecl 中的 ';' |
| 缺少右小括号')' | j | 报错行号为右小括号**前一个非终结符**所在行号。 | 函数调用 ( UnaryExp )、函数定义 ( FuncDef, MainFuncDef ) 、 Stmt 及 PrimaryExp 中的 ')' |
| 缺少右中括号']' | k | 报错行号为右中括号**前一个非终结符**所在行号。 | 数组定义 ( ConstDef, VarDef,FuncFParam ) 和使用 ( LVal ) 中的 ']' |
| printf中格式字符与表达式个数不匹配 | l | 报错行号为**'printf'**所在行号。 | 'printf' '(' StringConst {',' Exp}')' ';' |
| 在非循环块中使用break和continue语句 | m | 报错行号为**'break'与'continue'**所在行号。 | Stmt → 'break' ';'\|'continue' ';' |
| switch 语句中 case 和default重复 | n | switch 语句中出现了重复的 case 常量值，或者出现了多个 default 分支。报错行号为重复的 'case' 或 'default' 所在行号。 | CaseStmt → 'case' Number':' { Stmt }<br>\| 'default' ':' { Stmt } |

- 错误类别 $a$ 为词法分析中会出现的错误。
- 错误类别 $i$, $j$, $k$ 为语法分析中会出现的错误。
- 剩余错误类别均为语义分析中会出现的错误。



# 代码生成

- 考核目标代码的运行结果
  - PCODE：在解释执行程序上的运行结果
  - MIPS：用Mars运行的结果
  - LLVM IR: 用llc工具运行的结果
- 分两次作业
  - 先快速实现一个完整编译器
  - 再处理扩展的语法成分
- 应始终具备错误处理能力

# 代码生成

- 生成 PCODE 的编译器，生成 PCODE 并解释执行，在 pcoderesult.txt 中记录解释执行结果。
- 生成 LLVM IR 的编译器，将目标代码生成结果输出至 llvm_ir.txt 中。
- 生成 MIPS 的编译器，将目标代码生成结果输出至 mips.txt 中。

代码生成作业二以MIPS为目标码才能参加竞速排序

# 代码生成

testfile.txt

```
int test;
int main(){
   printf("Hello World\n");
   test = getint();
   printf("%d",test);
   return 0;
}
```

input.txt

2026

pcoderesult.txt或标准输出

Hello World
2026

# 竞速排序

- 运行结果正确
- 对每个文件计算FinalCycle = DIV*18+MULT*3 + JUMP/BRANCH * 1.5 + MEM * 2 + OTHER * 1的值，在运行正确的前提下，FinalCycle越小排名越靠前
- 每个文件根据FinalCycle的值分段计分
- 多个文件则对每个文件的得分加权求和
- 本次作业的前提是代码生成作业二需要以MIPS为目标码

# 竞速排序

## 评测结果

学生编译器编程语言
**Java**

目标代码类型
**MIPS**

## 测试用例

| 测试点名称 | 评测结果 | 性能赋分得分比例 | 指令周期数 |
|---|---|---|---|
| testcase1 | Accepted | 100% | Divide (18): 0 Multiply (3): 2013 Memory (2): 3005 Branch (1.5): 2001 Jump (1.5): 1501 Other (1): 33071 Final Cycles: 50373.0 |
| testcase2 | Accepted | 100% | Divide (18): 0 Multiply (3): 0 Memory (2): 55 Branch (1.5): |
| testcase3 | Accepted | 100% | Divide (18): 0 Multiply (3): 36 Memory (2): 374 Branch (1.5 |
| testcase4 | Accepted | 100% | Divide (18): 0 Multiply (3): 55 Memory (2): 464 Branch (1.5 |
| testcase5 | Accepted | 100% | Divide (18): 0 Multiply (3): 0 Memory (2): 28935 Branch (1 175729.0 |
| testcase6 | Accepted | 100% | Divide (18): 0 Multiply (3): 117724 Memory (2): 652253 Br Cycles: 7201569.5 |
| testcase7 | Accepted | 100% | Divide (18): 7776 Multiply (3): 15264 Memory (2): 98871 E 1102426.5 |
| testcase8 | Accepted | 100% | Divide (18): 26 Multiply (3): 95873 Memory (2): 145854 Br Cycles: 1823962.5 |
| testcase9 | Accepted | 100% | Divide (18): 1276 Multiply (3): 3672 Memory (2): 54196 Br 388451.0 |
| testcase10 | Accepted | 100% | Divide (18): 0 Multiply (3): 2000 Memory (2): 2002 Branch 31537.0 |

## 评测结果

学生编译器编程语言
**CPP**

目标代码类型
**MIPS**

## 测试用例

| 测试点名称 | 评测结果 | 性能赋分得分比例 | 指令周期数 |
|---|---|---|---|
| testcase1 | Accepted | >= 0% | Divide (18): 0 Multiply (3): 23000 Memory (2): 310048 Branch (1.5): 2001 Jump (1.5): 16504 Other (1): 137043 Final Cycles: 853896.5 |
| testcase2 | Accepted | >= 80% | Divide (18): 0 Multiply (3): 60 Memory (2): 112 Branch (1.5): 55 Jump (1.5): 210 Other (1): 1216 Final Cycles: 2017.5 |
| testcase3 | Accepted | >= 60% | Divide (18): 0 Multiply (3): 56 Memory (2): 545 Branch (1.5): 161 Jump (1.5): 379 Other (1): 2061 Final Cycles: 4129.0 |
| testcase4 | Accepted | >= 80% | Divide (18): 0 Multiply (3): 55 Memory (2): 1150 Branch (1.5): 413 Jump (1.5): 998 Other (1): 4867 Final Cycles: 9448.5 |
| testcase5 | Accepted | >= 80% | Divide (18): 0 Multiply (3): 0 Memory (2): 27806 Branch (1.5): 28246 Jump (1.5): 44672 Other (1): 292473 Final Cycles: 457462.0 |
| testcase6 | Accepted | >= 0% | Divide (18): 0 Multiply (3): 134588 Memory (2): 14241063 Branch (1.5): 690603 Jump (1.5): 704411 Other (1): 6030508 Final Cycles: 37008919.0 |
| testcase7 | Accepted | >= 0% | Divide (18): 7776 Multiply (3): 26784 Memory (2): 3488965 Branch (1.5): 34167 Jump (1.5): 82787 Other (1): 1469913 Final Cycles: 8843594.0 |
| testcase8 | Accepted | >= 0% | Divide (18): 26 Multiply (3): 123548 Memory (2): 3584096 Branch (1.5): 86129 Jump (1.5): 78095 Other (1): 1820702 Final Cycles: 9606342.0 |
| testcase9 | Accepted | >= 80% | Divide (18): 1696 Multiply (3): 6504 Memory (2): 53053 Branch (1.5): 22063 Jump (1.5): 36232 Other (1): 435235 Final Cycles: 678823.5 |

# 任务及考核说明

- 每次任务对应教学平台中1-4道作业，作业在第2周全部打开，随理论课进展依次关闭
- 每次作业请严格按照输入输出的要求实现，以便准确评判
- 提交后自动评判，若有错误可修改后再次提交
- 作业关闭前可多次提交，以最后一次结果为准
- 作业关闭后再提交会扣分，每晚交24小时，扣10%
- 期中和期末上机考核内容包括现场修改程序、新的测试程序、回答问题等，全程断外网，不能使用大模型
- 现场答辩，讲解自己的实现方案，并针对代码回答问题

# 评测及开发环境

- Clang12.0.0
- Java JDK 17
- Mars 2026(编译专用)
- Clion 2019.3.6
- Idea 2022.3.2
- 可以使用Cmake 3.28.3 进行项目管理

# 任务及考核日程

| | 一 | 二 | 三 | 四 | 五 | 六 | 日 | | |
|---|---|---|---|---|---|---|---|---|---|
| | | | | | | 建议提交作业 | 常规任务 | 考核任务 | 截止作业 |
| 第2周 | 9.14 | 9.15 | 9.16 | 9.17 | 9.18 | 9.19 | 讲解实验内容。发布讲座、所有实验作业打开 | | 9.20 |
| 第3周 | 9.21 | 9.22 | 9.23 | 9.24 | 9.25 | 文法解读提交 9.26 | | | 9.27 |
| 第4周 | 9.28 | 9.29 | 9.30 | 10.1 | 10.2 | 词法分析提交（免上机） 10.3 | | | 文法解读 10.4 |
| 第5周 | 10.5 | 10.6 | 10.7 | 10.8 | 10.9 | 补9月27日课，上机答疑 10.10 | 上机、答疑 | | 词法分析 10.11 |
| 第6周 | 10.12 | 10.13 | 10.14 | 10.15 | 10.16 | 语法分析提交（免上机） 10.17 | 上机、答疑 | | 10.18 |
| 第7周 | 10.19 | 10.20 | 10.21 | 10.22 | 10.23 | 语义分析提交（免上机） 10.24 | 模拟上机考核 | | 语法分析 10.25 |
| 第8周 | 10.26 | 10.27 | 10.28 | 10.29 | 10.30 | 10.31 | 期中上机考核 | | 11.1 |
| 第9周 | 11.2 | 11.3 | 11.4 | 11.5 | 11.6 | 11.7 | 上机、答疑 | 语义分析分享/现场考核 | 语义分析 11.8 |
| 第10周 | 11.9 | 11.10 | 11.11 | 11.12 | 11.13 | 代码生成一提交 11.14 | 上机、答疑 | 语义分析分享/现场考核 | 11.15 |
| 第11周 | 11.16 | 11.17 | 11.18 | 11.19 | 11.20 | 代码生成二提交 11.21 | 上机、答疑 | 代码生成现场考核 | 11.22 |
| 第12周 | 11.23 | 11.24 | 11.25 | 11.26 | 11.27 | 11.28 | 上机、答疑 | 代码生成现场考核 | 代码生成一 11.29 |
| 第13周 | 11.30 | 12.1 | 12.2 | 12.3 | 12.4 | 12.5 | 上机、答疑 | 代码生成现场考核 | 12.6 |
| 第14周 | 12.7 | 12.8 | 12.9 | 12.10 | 12.11 | 竞速排序提交 12.12 | 上机、答疑 | 代码优化现场考核 | 12.13 |
| 第15周 | 12.14 | 12.15 | 12.16 | 12.17 | 12.18 | 12.19 | 上机、答疑 | 代码优化现场考核 | 代码生成二 12.20 |
| 第16周 | 12.21 | 12.22 | 12.23 | 12.24 | 12.25 | 补10月4日课，代码优化现场考核 12.26 | 模拟上机考核 | | 竞速排序及其他（全部截止） 12.27 |
| 第17周 | 12.28 | 12.29 | 12.30 | 12.31 | 1.1 | 1.2 | 期末上机考核 | | 1.3 |

# 作业列表

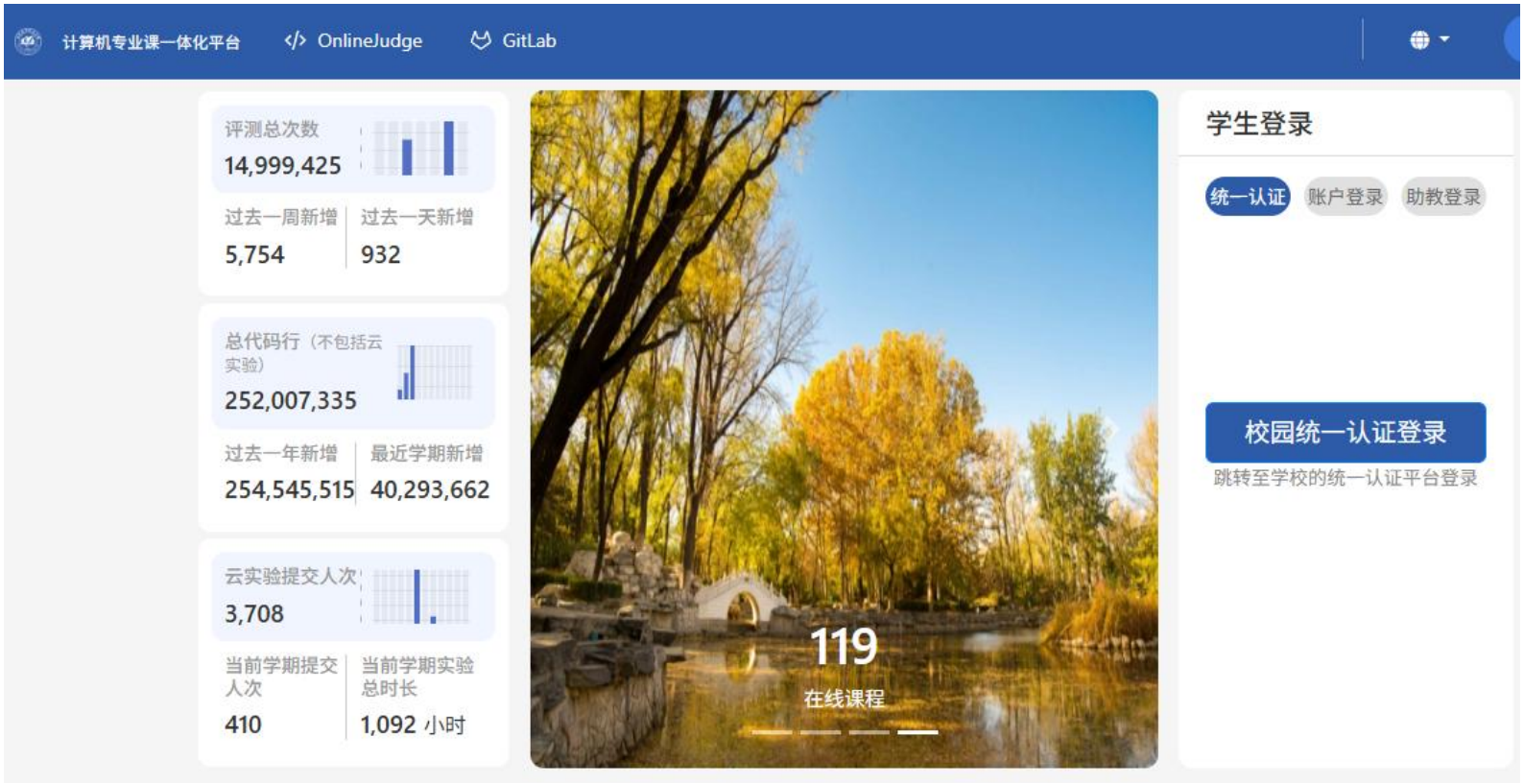
1. 2026文法解读（文法解读、AI交互记录）
2. 2026设计文档
3. 2026词法分析（词法分析、 AI交互记录）
4. 2026语法分析（语法分析、 AI交互记录）
5. 2026语义分析（语义分析、 AI交互记录）
6. 2026代码生成一（代码生成一、AI交互记录）
7. 2026代码生成二（代码生成二、AI交互记录）
8. 2026竞速排序（竞速排序、优化文章、优化前后中间代码目标代码、AI交互记录）
9. 2026总结感想

# 上机安排

- 第2-17周，新主楼F327、F332、F333、F334机房
- 上机时间可以自带笔记电脑到机房做实验作业并答疑
- 期中考核前，在建议提交作业时间之前提交指定作业并达到100%正确率，可免去下一次建议提交作业之前的上机
- 期中考核后，若在指定时间之前提交指定作业并达到100%正确率，即可参加现场考核，考核后可免去下一次作业现场考核之前的上机
- 第8周和第17周分别进行期中考核和期末考核，届时需要同学们到机房上机考核，会提前安排用于熟悉环境的模拟考试，具体安排另行通知

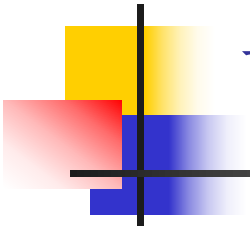
# 作业提交、课程信息获取

- https://judge.buaa.edu.cn/
- 帐号为学号，以前的密码或统一认证登录



# 实验平台

- 课程有关的材料均从实验平台中获取
- 所有实验作业都在实验平台发布、提交
- 课程公告会在实验平台发布
- 到实验平台答疑论坛提问或查找答案

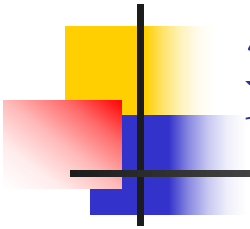


# 课程资料

- 课程信息-教学资料
  - 文法定义及相关说明
  - MARS2026
  - 编译器示例代码
    - PL/0
    - Pascal-s
    - SysY-Java版
    - SysY-Cpp版
  - 中间代码输出格式说明
  - 竞速排序及MARS仿真器说明
  - WSL使用指南
  - 公共测试程序库（一周后发布）
  - 专题报告PPT

# 课程资料

- 慕课
  - 词法分析专题报告
  - 语法分析专题报告
  - 语义分析专题报告
  - 代码生成专题报告
  - 代码优化专题报告
- 在线教程
  - 实验教程（2026）
  - 实验教程（Java版）
  - 实验教程（cpp版）



# 实验教程

- 介绍编译器开发完整流程
  - 前端部分：词法分析、语法分析、语义分析
  - 中间代码：LLVM、四元式、Pcode
  - 代码优化：寄存器分配、SSA……
  - 目标代码生成：MIPS
- 旨在为编译器的架构设计和实现提供参考
- 提供cpp版的示例编译器 tolangc
  - 项目地址：https://github.com/wokron/tolangc

# 能得到的帮助

- 理论课堂
- 课程助教
- 实验教程
- 大模型

# 对大模型的使用要求

- 可以使用大模型辅助生成代码
  - 需提交与大模型的交互记录（有专门的题目）
  - 设计文档里需写出对生成代码的评价和对代码的人工修改
- 鼓励"手搓"代码
- 设计文档、申优文章和总结感想不能用大模型生成，若发现会影响实验成绩
- 上机考核期间不允许使用大模型
- 现场答辩期间不允许使用大模型

# 对大模型的使用要求

- 将大模型作为辅助工具使用
  - 辅助学习和理解理论知识，辅助解决课程作业的难疑点，增强对理论知识的掌握
  - 辅助编程和编译器开发，设计更好的编译器架构，学习编译优化方法
- 保持自己的思考习惯
  - 能判断大模型输出结果的好坏、正误
  - 能读懂、解释大模型生成的代码
- 保持自己的编程能力
  - 能识别出代码中的问题，并对有问题的地方进行修改
  - 能根据新需求修改代码

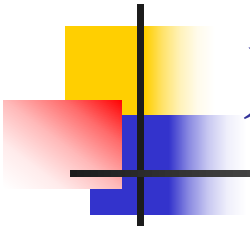
# 查重及作弊有关

- 出现项目代码整体相似度过高，或其他高度相似的情况的，视为代码相似。
  - 抄袭者与被抄袭者同等处理
  - 被查出代码相似的学生有权提出申诉
- 以任何非正当方式获取测试用例、直接输出结果的，均视为作弊行为。
- 编译技术课程组保留关于作弊行为的解释、调查和处置的权力。
- 提醒注意：
  - 不得直接照搬公开代码库
  - 妥善保管代码，避免泄露
  - 建议使用 Git 等工具记录开发过程

# 交流与沟通

- 现场答疑
  - 每次上机时间
- 交流
  - 组织专题分享、讨论
  - 在论坛上发起话题在线讨论，提问前先确认是否有同样的问题已解决
  - 汇总常见问题发布在论坛
  - 请助教帮助解决代码问题前，先用公共测试程序库进行调试
- 联系方式

杨海燕：compiler_buaa@126.com
82317624 G908
史晓华：xhshi@buaa.edu.cn



# 为什么会觉得难

- 编译原理本身没弄清
- 需要综合运用多门课知识
  - C/C++/JAVA语言、数据结构、算法、汇编
- 编程经验不足
  - 从头开始做
  - 规模相对较大
  - 调试困难
- 需要自己独立完成

# 你该怎么做

- 1. 学习、复习有关知识
- 2. 确定适合自己的难度
- 3. 跟上各阶段步骤，按时完成阶段作业
- 4. 及时做，自己做、坚持做
- 5. 积极交流、沟通

# 你能得到什么

自己动手做了才会有收获
自己动手做了一定有收获

编译原理本身没弄清

需要综合运用多门课知识

编程经验不足

需要自己独立完成

耐力

毅力

体力

巩固了编译原理各个知识点
能够构造完整的编译器

复习了多门课知识
学会了知识点在编译器及其构造中如何应用

积累了一定的编程经验
开发了一个具有一定规模的完整系统

提高了独立解决问题的能力
遇到问题能设法解决，激发创造性、探索能力

时间管理

.....

**想，都是问题，做，才是答案**

# 特别帮助

尽管大家在之前的课程中积累了许多编程经验，但是独立设计一个编译器仍可能是不小的挑战，最终的代码量可能有上万行，因此非常考验大家的编程能力和项目管理能力。

**需要帮助？**

- 觉得自己独立完成编译器有困难？
- 没有思路，无从下手？
- 很难通过评测题目？

**帮助别人！**

- 学有余力，对完成编译器很有信心
- 乐于分享自己的学习经验
- 愿意抽出时间帮助其他同学

**联系我们**

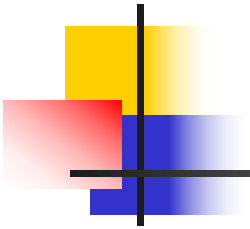
不论你需要帮助，还是想加入我们帮助其他同学，随时联系自己班级的助教，我们会尽可能地提供全面的帮助！

# 特别提醒

- 1. 选定题目，坚持到底，慎重换题
- 2. 了解各项要求，紧跟阶段步骤
- 3. 有疑问，及时沟通，设法解决
- 4. 相信自己，展示实力
- 5. 立足于自己思考，不要过度依赖参考文档和LLM
- 6. 成绩公布前，务必自留作业备份
- 7. 确认选课（教务系统、教学平台）
- 8. 期中考试前需要用机房的机器至少上机一次，确保代码能在机房环境下运行、调试

# 特别提醒

- 独立完成！！！
  - 测试程序
  - 代码
  - 文档
- 不能用提交作业代替全面测试
- 误操作需自负后果
- 作业具体要求、参考资料以教学平台发布的为准



# 参考资料

- 《编译技术》第17章 第18章及PL/0、Pascal-s编译器源代码
- 参考书
  - Compilers: Principles, Techniques, and Tools. By Alfred V. AHO, Ravi SETHI and Jeffrey D. ULLMAN
  - 中文版：编译原理，李建中，姜守旭译，机械工业出版社
    编译原理，赵建华，郑滔，戴新宇译
  - Advanced Compiler Design and Implementation. By Steven S. Muchnick.
  - 中文版：高级编译器设计与实现，赵克佳，沈志宇译，机械工业出版社

# 总结感想摘录

这三个月写编译器，最大的感悟就是设计大于实现。

我不记得重构了多少次：语法分析重构了至少两次，错误处理后重构了文件结构，写完LLVM又把整个Value体系重构了一遍。在此之前，代码里到处都是特判和乱飞的字符串，看着就头大。

如果有机会再写一次编译器，我一定会：

1. 一开始就设计好Value继承体系
2. 先把SSA/Mem2Reg的原理搞清楚再动手
3. 写更多的单元测试

写编译器的日子里，不知道熬了多少个夜，感谢每一位答疑的助教，感谢课程组提供的测试工具。

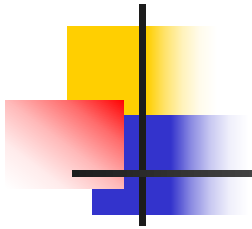
总之，编译无疑是我这学期投入最多、也最有成就感的一门课。虽然最终的编译器还很稚嫩，优化也做得很保守，但从零开始写出一个能跑的编译器，这种感觉真的很棒。

经过一个学期的编译课程学习，我对编译器这一系统软件有了从模糊印象到整体理解、再到实际动手实现的完整认识。这门课带给我的收获，不仅是具体的技术知识，更重要的是一种看待复杂系统的思维方式。

在学习这门课之前，我对编译器的理解更多停留在“把高级语言翻译成机器能执行的代码”这样一个抽象层面，对其中的具体过程几乎没有概念。本学期通过系统地学习词法分析、语法分析、语义分析、中间代码生成以及优化和目标代码生成等内容，我第一次真正理解了“程序是如何一步一步被理解、被约束、被转换”的。尤其是在自己动手实现 SysY 编译器之后，很多原本在课堂上看似枯燥、抽象的概念，比如AST、符号表、作用域、控制流图、SSA 形式等，都变成了具体而清晰的代码。

对我来说，收获最大的一点是：编译器不是“魔法”，而是由一系列明确规则和工程取舍堆叠出来的系统。

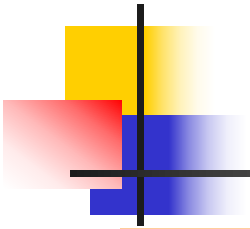
真正动手实现编译器之前还一直在怵头，这么庞大又复杂的项目我真的能独立做出来吗？好在课程组提供了一个编译器项目供参考，让我的开始变得不那么困难，可以看到一个编译器项目具体是怎么编排的对后面具体写起代码来非常有帮助。在实现过程中，我多次体会到“理论上很优雅，但实现时并不简单”的情况，比如错误恢复、短路求值、寄存器分配以及地址取用等问题。这些问题在 PPT 上往往只是几页纸，但在实际代码中却需要反复调试、推翻重来，这让我真正理解了课程中很多设计决策背后的现实原因。



我觉得编译这门课一个最大的好处就是理论和实验关系非常紧密，我们实验课上实现编译器的脚步完全可以跟着理论课来，比如：我们在理论课上学到使用有穷自动机实现词法分析，课下我们可以立刻模仿理论课件中给出的伪代码完成实验课中词法分析一节，理论课上讲到的优化方法我们可以应用到自己的编译器上提升优化效果。理论的学习带来实验实现上的轻松，实验应用理论课的知识又加深了对理论课相关知识的理解。

其次，编译课教会了我提前规划，虽然是通过让我吃苦头来教会我的。实验部分简直就是“一头披着羊皮的狼”，前半学期的词法分析、语法分析、语义分析部分和后半学期的代码生成和优化简直不是一个难度的。前半学期的实验都能早早的完成，然后无所事事，后半学期直接开始上强度，代码量激增，完成一次作业所需时间大幅增加。现在来看，前期空出来的时间就应该抓紧写数据库大作业和学习理论课，这样就不会像我一样在12月份被编译理论考试、赶数据库大作业、编译实验课收尾三座大山压着了。我应该在实验开始之前就开始规划实验的进度，在规定的时间内完成相应的内容。

实验课还提高了我debug的能力，相比于CO和OO，编译实验课debug难度大幅增加，我们不只是要de自己代码的bug，还要de生成出来的代码的bug，最难的是做代码优化的时候，如果做完优化竞速排序出现了bug，我们要找llvm和mips两个方面的bug。不过，我也学会了使用AI大模型辅助找bug，然后让它给出测试样例，修bug的速度大大加快。



编译课程从九月底快国庆的时候开始编写编译器的词法分析，到十二月初完成中间代码生成，不知不觉就写了这么多代码，感觉时间过得很快，每次都是在截止前的第二周开始编一个星期的代码，空一个星期又开始赶下个阶段代码，写着写着就写完了（留给我们的作业时间还是很宽松的）。经过这一过程，我对编译过程了解更深了。刚开始听到编译时上大一写 C 语言，在大二的计组老师也用了一些时间讲编译的过程，总是似懂非懂，编译上机让我真正理解了编译。从 0 写到生成 LLVM 给了我很大的成就感，让我用 java 用的更熟练了。课程最后留了三条路供我们取舍，给我这种水平不高的选手留了很好的后路。另外，编译这门课的上机和课内知识联系的非常紧密，课上讲到如何实现，课下的代码就能如何实现。课程组给的指导手册也很好，给了我很多方向性的提示，对我帮助很大。课程组的测试库给我的帮助最大，让我充分感受到了调试的乐趣，完整的测试库给我一种只要代码能跑就能调试成功的感觉，虽然有时连续长时间调试好几天，但我还是很有干劲。编译课的刘老师讲的虽然很慢，但是讲的很细致，讲的非常非常清楚。总的来说，编译课是我经历的非常不错的一门课。

打开LLVM教程，我才意识到要完成的工作有多复杂。如果以教程中的方式做，我要先花时间弄明白教程的逻辑，再构建项目、写代码。我先尝试理解教程的逻辑，但是因为对LLVM完全不熟悉，教程的内容很难理解。我又去学习LLVM的语法，这时候时间就所剩无几了，我只能一边看语法一边尝试写一些生成代码。我决定同样按语法类型区分方法，一种类型使用一种方法，这样能够更好的把握进度。对于其他语法类型的翻译都比较简单，条件语句短路求值和标签跳转理解逻辑之后生成跳转语句也不太困难，写到这里的时候已经到了代码生成一的最后一周了，所有代码生成一二的内容我没有分开写。这段过程的时间主要花在了定义和符号表上了，一直在小修小补，修改符号表的逻辑和定义的命名方式以及记录定义的命名，好在最后在代码生成一截止前完成了任务。

这个时候其他课也有很多事要忙，花了很多时间完成代码生成之后，我又去忙其他课的事情，再加上中间代码生成结构的问题，我已经没有时间和精力把LLVM翻译成MIPS了，只能草草收场，以LLVM作为最终代码。

从一开始感觉时间宽松游刃有余，到后面自顾不暇手忙脚乱，有些遗憾也有些无奈。如果学期初能知道要面对什么，是否结果会不一样?

编译实验做完了（好吧其实也没做完，代码生成二最后只通过了 76%），回头看看这一路，感触挺深的。

开始的时候觉得没那么难，词法分析就是识别单词，语法分析建棵树，想着快点写完就行。代码写得挺随意，怎么方便怎么来。到了语义分析要填符号表的时候，问题就来了——前面图省事，现在数据结构不好用，接口也得改，结果花了好多时间在重构上，真成了“欠债式开发”。这时候才痛彻地明白，实验要求做的总体设计文档真不是摆设。好的设计不是给老师看的，是给自己后面省力的。没个清晰的蓝图，代码越多越是一团乱麻。

现在回头看，这门课的收获远不止“实现了一个编译器”这么简单。它让我明白了，那些我们平时习以为常的编程语言特性，背后都有一套复杂而精妙的实现逻辑；也让我学会了面对复杂问题时，如何拆解成一个个小任务，逐个突破。更重要的是，它让我找回了面对难题时的耐心和韧性——从“卡壳到崩溃”到“冷静排查”，从“毫无头绪”到“主动查资料、问老师同学”，这个过程中的成长，是任何理论课都给不了的。

虽然实验过程充满了煎熬和挫败，但当我把最后一个测试用例成功编译并运行时，所有的辛苦都变成了值得。这门编译实验课，不仅提升了我的技术能力，也让我对项目编程有了更深的敬畏和热爱。正是通过这样一次次挑战极限的实验，才能慢慢积累起来实力，这些是vibecoding给不了我们的。

如果你问本鹅这门课难不难，本鹅会说：难，真的难。但也正因为难，完成之后的成就感才会那么强烈。

编译器是一个把高层抽象翻译成底层实现的工具，而做编译器的过程，也是一个让自己从"用代码"变成"理解代码"的过程。当你知道一行简单的 a = b + c 背后经历了词法分析、语法分析、语义检查、IR生成、优化、寄存器分配、指令生成这么多步骤时，你看代码的眼光会完全不同。

这个项目本鹅写了几千行Java代码，踩了无数的坑，熬了无数的夜。但现在回头看，一切都值得。

希望这篇碎碎念能帮到同样在编译原理这条路上挣扎的你。记住：代码能跑是第一步，跑得对是第二步，跑得快是第三步。一步一步来，总会到达终点的。

上了本学期的编译课实验课程，我收获很大

1. 首先就是不要低估编译器开发的难度，课前我只以为这个编译器开发只是一组文本到文本的翻译游戏，遵循语法规则，但是实际开发过程中各种bug层出不穷，而且相比于之前的程序开发，编译器开发debug的时候还需要根据生成代码的bug反推，但是这种打破传统开发debug的经历让我反思debug本身，而不是无脑找bug，复现之后修改与预期错误的部分

2. 其次，我深刻体会到了测试工程的重要性，公开样例往往不足以帮助我改正代码，这时，测试工程的构建就变得尤为重要，我深刻研究了样例的构造，而这也在后续做完优化之后检查正确性上给了我很多帮助

3. 最后，我意识到了架构的重要性，好的架构在开发过程中有非常大的帮助

重新回看这学期自己的实验，最大的感想是：本人极为善于选择最困难的路。归根结底是，本人总是不善于需求帮助，有些闭门造车的意味，不懂得借鉴已有的想法，而总是想着自己从轮子造起；而且没有对于整个项目的长远眼光，没有进行到那一步的时候总想不清应该用哪一种方式处理当前阶段的数据，给出怎样的输出，只能短视地根据当前要求的输出设计模块，于是无可避免地在一次次迭代中重构了自己代码。最印象深刻的便是代码生成那部分，盲目地一头撞进四元式的设计里，最后一无所获。所幸，助教们不离不弃，鼓励着本人继续完成实验，并耐心地给出了相应的指引，最后本人用了一周左右的时间完成了 LLVM-IR 的生成，悔不当初又如释重负。

“在做实验作业的过程中遇到任何困难或问题请及时联系老师或助教，课程组将为同学们尽力提供帮助。如果因为各种原因导致作业进展比较慢，也请坚持做下去。”

当时因为各种原因，懒惰也有、迷茫也有、各课程压力也有，总而言之实验任务迟迟没有完成，错过了代码生成1，眼看着代码生成2也要错过了，真的非常焦虑，濒于自暴自弃的边缘，看到课程组发的这段话时，真的觉得很暖心，又觉得可以再坚持坚持😭。虽然最后还是止步于代码生成2，但至少可以参加期末，也应该不会挂科了，真的很感谢助教们的帮助和鼓励！🫶🫶

编译实验对我来说非常具有挑战，但同时也让我的工程能力和代码思维得到了极大的提升。从词法分析开始一步步构建，每一次增量开发我都需要兼顾已有部分，并考虑后续内容，思考如何设计和编写能够使得代码更易于维护，更不易出错。

随着代码量的逐步增加，我愈发感受到了压力，做到代码生成时，我曾因为一次又一次得到错误结果的提交而几近崩溃：代码量太大，存在细节问题的地方太多，构造的样例又无法测试出错误。这样的压力也迫使我一点点提高对一个代码工程的管理能力和掌控能力。在不懈的坚持和尝试下，我还是顺利完成了mips生成的编写。

但在代码优化中还是遇到了不少困难。在较为复杂、改动较大的优化方法的实现中，我发现已有的代码结构并不能很好的适应新增的优化，导致工作量变大、debug难度增加。最终并没有很好地完成优化任务。

编译实验让我第一次自己完成了一个较大的代码项目，让我意识到在复杂工程中，良好的规范和规划起着关键性的作用。比如在进行语义分析前应当想好符号表如何组织，在写最后的mips代码生成前想好寄存器、内存管理的处理方式和文件结构。这些设计能让编写代码时思路更加清晰，避免写了大半发现一片混乱而不得不重构的情况，提高开发效率和质量。

刚刚拿到实验要求时我一头雾水，根本读不懂SySY语言的定义以及复杂的语法规则。经过测试程序的编写、联系理论课程对语法规则进行解读，我才逐渐了解了要做的任务，这个过程锻炼了我理解需求的能力在设计架构时我的系统管理能力得到了提升。面对这样一个庞大的工程，我知道如果不进行系统的设计，那么最后一定会变成屎山代码。因此在具体实现代码之前我先看了课程组提供的参考实现与往届同学的实现，了解了编译器实现的基本架构，比如前中后端放在不同的包里、集中定义enum枚举类来提升代码可读性等，再参考他们的架构来实现，这让我更深入的体会到了一个合理架构的重要性编写具体实现提升了我对于面向对象编程的能力。虽然上个学期在OO课程中学习了面向对象程序设计，但我之前没用Java写过如此庞大的工程。在具体的实现过程中，我在很多地方也使用到了之前学到的面向对象的方法，例如对于所有错误创建一个单例模式来管理、使用工厂模式管理1位/32位int、利用中间类（比如ScopeManager）来对作用域复杂的管理进行封装降低复杂度、创建分发器对于不同类型执行不同处理等，这为我面向过程语言开发积累了更多经验。

对生成代码进行测试与Debug的过程提升了我的测试思维。一开始我习惯全部写完代码再整体测试，后来发现这会导致一开始面对全是错误/根本跑不起来的代码我从下手。后来我学会了写一个部分就对一个部分进行单独测试，这样大大减少了Debug压力。在面对整体的错误时，我也会尝试使用分段处理的原则对bug进行定位，例如去掉这个优化还报不报错，使用这个简单的实现替换这一部分代码还有没有问题等。

我的测试思维在这个过程中得到了提升。

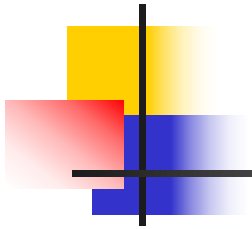
通过对于编译器的编写，我又完成了一项复杂的代码工程，我的开发能力又上了一个台阶。

拖了好久的感想文档，回想起这学期的编译课程，确实像胡院长所说，是人生第一次完成这么大的项目，其实之前的软工项目，代码量也不小，能达到上万行，但那是在成熟的框架上的组装，而编译确实是从编写样例开始的纯手搓，依靠的就是每周的迭代。
不愧是传说中本科阶段top1的课，难度来自对理论的深度理解，包括强硬的代码基本功，还有高压的期中、实验、竞速、期末的安排，个人感觉难度是大于os的，os改的代码量远远小于编译，只是上机考多了一些，本学期印象很深刻的就是在十月一出去玩晚上还在酒店写编译（）

在竞速之前遇到的问题绝大多数都能在评论区找到答案，答疑区就像是瑰宝，还能看到十多年前学长们的帖子，像是一种穿越时空的对话。但是竞速的盲debug实在是痛苦，卡的最久最久的是tc6的WA 50%，评论区也没有找到问题，后面在wfy助教的帮助下才解决问题，是形参与实参的混淆问题，多谢wfy助教
第一点是理论与实际的联系，编译这种“所学即所用”的感觉非常震撼，让我明白那些枯燥的算法是如何在工业界落地的。这是我大学以来第一次觉得理论课如此实用，对我的科研任务也有很大帮助。

第二点是代码基本功的进步，面对庞大的 AST 节点和复杂的 IR 指令体系，如果继承关系和多态设计得不好，后期维护简直是灾难。我被迫学会了如何使用抽象类和接口来解耦。

最后一点就是心态上的变化，竞速tc6改的想跳楼，实在实在是要崩溃了，再加上期末复习，真的绝望的一批，后来在助教的帮助下，再加上迅速调整心态，也算是客服了困难，这种经历在以后的道理上是帮助极大的



最大的收获可能不是学会了多少算法或者设计模式，而是真正理解了一个完整的编译器乃至软件系统是怎么搭建起来的。以前写代码可能就是一个文件、几个函数，但这次是一个有明确阶段划分、模块之间要互相配合的工程。比如前端生成的 AST 要交给中端去建符号表、查语义错误，中端生成的 IR 又要交给后端去翻译成汇编，这种“流水线”的感觉让我对软件架构有了更具体的认识。还有就是调试能力真的提升了太多，因为编译器一旦出错，往往不是一句报错就能定位的，经常要一步步看输出文件、对比中间结果，有时候甚至要手动模拟执行流程，这个过程虽然繁琐，但特别锻炼耐心和逻辑思维。

值得一提的是，该课程“禁止AI生成文档”的规定，反倒成为整个项目中最具价值的环节。撰写文档时，我不得不系统梳理整个学期的项目代码——那些冗余的函数、被注释的失败尝试、AI生成后未删除的定义，以及我零散的思考与实践痕迹，都在逐字撰写的过程中逐渐清晰。若允许AI生成文档，我大概率无法静下心来复盘这一切，也难以真切表达对项目价值的质疑，最终只会产出一份千篇一律、缺乏思考深度的“格式化作业”。正是这份强制的“手写反思”，让我跳出“完成项目任务”的表面桎梏，真正开始思考：我们完成项目的核心意义，究竟是“产出成果”，还是“在过程中实现成长”？

在开启编译实验之旅前，我便早已听闻这门实验的 “恐怖传说”：动辄数千行甚至上万行的代码量、无休止的重构返工、临近期末时编译优化任务与其他课程大作业的集中轰炸，每一项都让人心生敬畏。因此，从实验伊始，我就不敢有丝毫懈怠，始终保持高度重视，作业一经发布，便第一时间投入知识点学习、思路构思与编码实践，生怕落后进度陷入被动。实验前半阶段的词法分析与语法分析，因任务目标相对明确、逻辑链路较为清晰，我完成得还算顺利。但也正是这份 “顺利”让我犯了急躁的毛病，忽视了老师开篇就强调的整体架构设计要求，没有深入研读参考代码并规划编译器的全局结构，仅满足于完成当下的实验任务。比如设计词法分析的接口时，未考虑后续语法分析的调用需求；定义 AST 节点时，也未预留语义分析所需的属性字段。这种短视的做法，在进入语义分析、代码生成乃至优化阶段后，隐患集中爆发，频繁出现接口不兼容、数据结构适配性差的问题，我不得不反复回头重构前期代码，原本以为的 “捷径”反而变成了阻碍，耗费了大量本可避免的时间与精力。这也让我深刻领悟到，架构设计对复杂项目而言至关重要，好的全局规划能让编码事半功倍，而缺乏长远考量的仓促编码，只会为后续发展埋下无数陷阱。此次实验最大的收获，莫过于实现了理论与实践的深度融合。课堂上抽象的编译原理知识，诸如词法分析的状态机模型、语法分析的递归下降算法、语义分析的语法制导翻译等，都在实际编码中得到了充分落地。当亲手用代码实现状态机识别词法单元，用递归下降逻辑构建抽象语法树时，我对这些理论知识点的理解不再停留于书本层面，而是形成了具象且深刻的认知，真正打通了理论与实践的壁垒。同时，超过五千行的有效代码量，让我切实体会到面向对象思想与软件工程方法的核心价值。编译器的设计天然契合面向对象的特性，比如 LLVM 中间代码生成中，各类指令均可继承自统一的父类，这种 “继承”特性极大简化了代码实现；单例模式、工厂模式等经典设计模式的应用，也让代码的管理与扩展更加有序。若不借助这些软件工程方法对代码进行规划，随着实验推进，代码量不断膨胀，整个编译器必然会变得杂乱无章，难以维护。此外，我也终于厘清了编译与运行的核心边界，明白了哪些工作需要在编译阶段完成，哪些则需留待程序运行时实现，对理论课上 “符号表在编译时建立、运行栈在运行时构建”的知识点有了透彻理解，对 “遍”的概念也从抽象认知转化为具象体验 —— 几乎每完成一个子任务，都需要对源程序或中间数据结构进行完整扫描，这种方式虽结构清晰，却也让我真切感受到了编译程序开发的工作量之巨。实验过程中，挑战与成长并行。我曾不止一次为了排查一个隐蔽的 bug 奋战到凌晨三四点，经历过代码提交后失败的郁闷，遭遇过明知有错却找不到问题根源的迷茫，也感受过熟读文法文档后发现疏漏并成功弥补的恍然大悟。每次想要放弃却又咬牙坚持，最终攻克难题时，那种畅快淋漓的感觉难以言喻。而当实验接近尾声，回头翻看自己写下的数千行代码，那种成就感如同赚到人生第一桶金般珍贵。